Solution

General remarks:

- The number of ★'s gives a rough indicator of the task's difficulty. You should aim to solve at least all tasks with at most three ★'s. Tasks with more stars can be hard or require mathematical background that you should have from previous courses.
- You can always ask us for feedback or help during the exercise class.
- We recommend that you first read all tasks during class. Make sure that you have a rough approach for every task in mind before you start working on the details. Ask for help if a task is unclear such that you do not get stuck when we are not there to help.

1 From GCL to Program Graphs (★)

Consider the GCL program

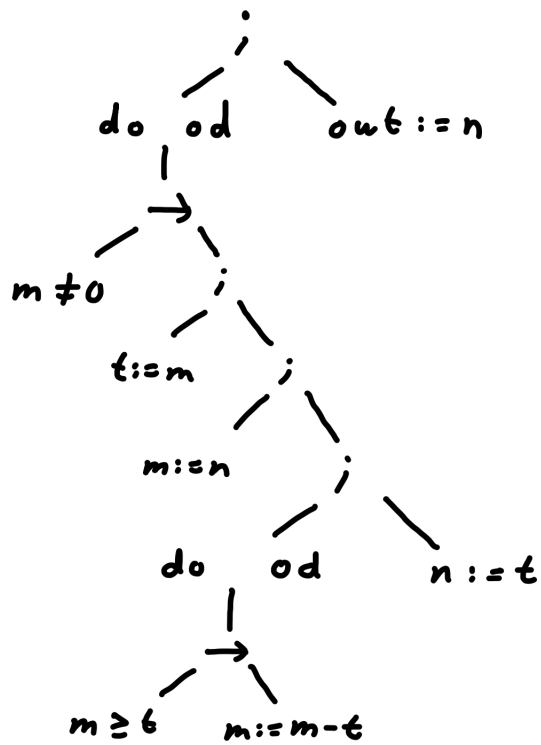
```
do m != 0 -> t := m;
             m := n;
             do m >= t -> m := m - t od;
             n := t
od;
out := n
```

Here, we write `m != 0` instead of $\neg(m = 0)$, `m >= t` instead of $m \geq t$, and `!b` instead of $\neg b$.

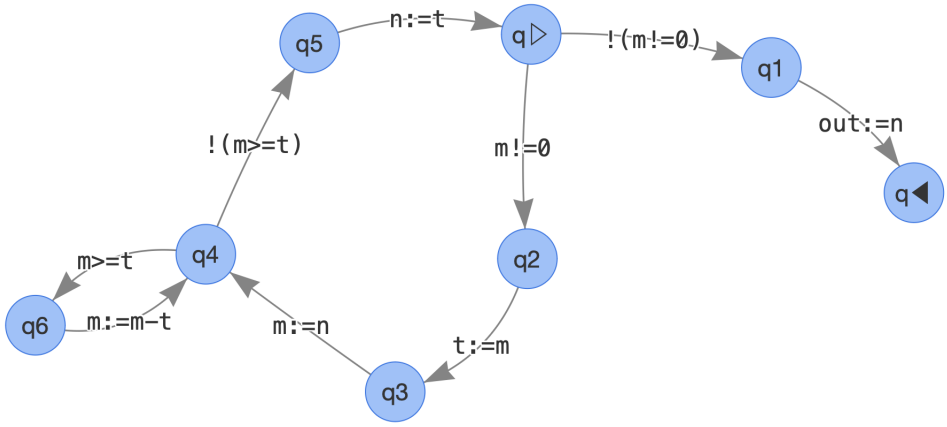
1. Construct an abstract syntax tree (AST) corresponding to the program.
2. Construct the corresponding program graph as described in the lecture.

Solution

1. Notice that there are different versions of correct ASTs (we may rotate around inner nodes). One possible solution is as follows:



2. We need to compute **edges**$(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\mathtt{C}]\!]$, where $\mathtt{C}$ is the Guarded Command program from above. The resulting program graph is as follows:


q5
n:=t
q▷
!(m!=0)
q1
out:=n
q◀
!(m>=t)
m!=0
q4
m>=t
q2
q6
m:=m-t
m:=n
t:=m
q3

2 More Expressions (★★)

1. Our arithmetic expressions a and Boolean expressions b are quite limited. Extend the syntax and semantics of expressions to also incorporate `false`, $a_1 \neq a_2$, $b_1 \vee b_2$, $a_1 \leq a_2$, $b_1 || b_2$, where $b_1 || b_2$ is a short-circuiting version of disjunction that does not evaluate b_2 if b_1 evaluates to `true`.

2. Do any of the added expressions increase the expressive power of the Boolean expressions or could we have managed without them? For example, $a_1 \neq a_2$ can be encoded as $\neg(a_1 = a_2)$. Can you encode the rest of the operations?

Solution

1. We first extend the syntax by extending the BNF grammar defining b:

$$
\begin{aligned}
b \ ::= \ & \ldots && \text{(all previous definitions from the book)} \\
| \ & \texttt{false} \\
| \ & a_1 \neq a_2 \\
| \ & b_1 \vee b_2 \\
| \ & a_1 \leq a_2 \\
| \ & b_1 \,||\, b_2
\end{aligned}
$$

Furthermore, we need to update the evaluation function $\mathcal{B}[\![b]\!]$ such that it incorporates the added atoms and composite expressions:

b	$\mathcal{B}[\![b]\!](\sigma)$
$\texttt{false}$	$\texttt{false}$
$a_1 \neq a_2$	$\begin{cases} \texttt{true}, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma), z_2 = \mathcal{A}[\![a_2]\!](\sigma), z_1 \neq z_2 \\ \texttt{false}, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma), z_2 = \mathcal{A}[\![a_2]\!](\sigma), z_1 = z_2 \\ \texttt{undefined}, & \text{if } \mathcal{A}[\![a_1]\!](\sigma) \text{ or } \mathcal{A}[\![a_2]\!](\sigma) \text{ undefined} \end{cases}$
$b_1 \vee b_2$	$\begin{cases} \texttt{true}, & \text{if } \mathcal{B}[\![b_1]\!](\sigma) = \texttt{true} \text{ or } \mathcal{B}[\![b_2]\!](\sigma) = \texttt{true} \\ \texttt{false}, & \text{if } \mathcal{B}[\![b_1]\!](\sigma) = \texttt{false} \text{ and } \mathcal{B}[\![b_2]\!](\sigma) = \texttt{false} \\ \texttt{undefined}, & \text{otherwise} \end{cases}$
$a_1 \leq a_2$	$\begin{cases} \texttt{true}, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma), z_2 = \mathcal{A}[\![a_2]\!](\sigma), z_1 \leq z_2 \\ \texttt{false}, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma), z_2 = \mathcal{A}[\![a_2]\!](\sigma), z_1 > z_2 \\ \texttt{undefined}, & \text{otherwise} \end{cases}$
$b_1 \,||\, b_2$	$\begin{cases} \texttt{true}, & \text{if } \mathcal{B}[\![b_1]\!](\sigma) = \texttt{true} \\ \mathcal{B}[\![b_2]\!](\sigma), & \text{if } \mathcal{B}[\![b_1]\!](\sigma) = \texttt{false} \\ \texttt{undefined}, & \text{otherwise} \end{cases}$

2. We could have managed without them because each of the added Boolean expressions can be defined as syntactic sugar in terms of the existing ones:

- $\texttt{false} \triangleq \neg\texttt{true}$
- $a_1 \neq a_2 \triangleq \neg(a_1 = a_2)$
- $b_1 \vee b_2 \triangleq \neg(\neg b_1 \wedge \neg b_2)$
- $a_1 \leq a_2 \triangleq \neg(a_1 > a_2)$
- $b_1 \,||\, b_2 \triangleq \neg(\neg b_1 \,\&\&\, \neg b_2)$

3 Arrays (★★★)

Incorporate operations for arrays into the Guarded Command Language. That is, extend the syntax and semantics of the Guarded Command language such that it supports assignment commands `A[a1] := a2` for writing to an array element and arithmetic expressions `A[a]` for reading the value of an array element.

Hint: To define the semantics of commands and expressions involving arrays, you also have to extend the definition of memories σ, such that they can assign values to array elements.

Solution

Note: Arrays have not been covered before FM lecture 3.

We first extend the syntax:

$$\begin{aligned} \texttt{a} &::= \ldots \mid \texttt{A[a]} \\ \texttt{C} &::= \ldots \mid \texttt{A[a1]:=a2} \end{aligned}$$

To define the semantics function $\mathcal{S}[\![\texttt{A[a1]:=a2}]\!]$ and the evaluation function $\mathcal{A}[\![\texttt{A[a]}]\!]$, we first incorporate arrays into our definition of memories. To this end, let **Arr** denote the set of arrays, and $\mathsf{size}(A)$ denote the size of array $A \in \textbf{Arr}$. The set of memories is then given by:

$$\textbf{Mem} \;=\; \{\sigma \mid \sigma\colon \textbf{Var} \cup \{A[i] \mid A \in \textbf{Arr}, 0 \leq i < \mathsf{size}(A)\} \to \textbf{Int}\}$$

$$\mathcal{S}[\![\texttt{A[a1]:=a2}]\!](\sigma) \;=\; \begin{cases} \sigma[\texttt{A[u]} \mapsto v], & \text{if } u = \mathcal{A}[\![\mathsf{a1}]\!](\sigma) \text{ defined,} \\ & \quad v = \mathcal{A}[\![\mathsf{a2}]\!](\sigma) \text{ defined,} \\ & \quad \texttt{A[u]} \in \mathsf{dom}(\sigma) \\ \text{undefined}, & \text{otherwise} \end{cases}$$

$$\mathcal{A}[\![\texttt{A[a]}]\!](\sigma) \;=\; \begin{cases} \sigma(A[u]), & \text{if } u = \mathcal{A}[\![\mathsf{a}]\!](\sigma) \text{ defined}, \texttt{A[u]} \in \mathsf{dom}(\sigma) \\ \text{undefined}, & \text{otherwise} \end{cases}$$